

React Interview Prep

Topic 16 — HOC & Render Props: Complete Guide

Gift Wrap Analogy · HOC Syntax · withAuth · withLoading · withLogger · HOC Rules · Render Props · Toggle · 3-Way Comparison · When to Use · Q&A;

- Covers All examples from conversation — nothing skipped
- Series React Interview Prep — Session 16

by Akshay Chavhan • React Interview Series

1. Like You're 5 — Two Analogies

■ HOC — Gift Wrapping Machine

You put a plain toy inside the machine.

It wraps it with paper, adds a bow, sticks a label.

Out comes a BETTER version of the same toy.

HOC = function that takes a component
and returns an ENHANCED version of it.

■ Render Props — Vending Machine

The machine has coins + buttons (logic).

But YOU decide what happens on button press.

You tell it: 'When I press snacks, show THIS UI'

Render Props = component provides logic,
YOU provide the UI through a function.

PART 1 — Higher Order Components (HOC)

2. HOC — Basic Syntax

```
// A HOC is just a function:
// Takes a component → returns a new enhanced component
function withSomething(WrappedComponent) {
  return function EnhancedComponent(props) {
    // Add extra behaviour here
    return <WrappedComponent {...props} />;
    // ↑ always spread original props!
  };
}
```

■ **HOC naming: always starts with 'with'. Returns a NEW component — never mutates the original.**

3. withAuth — Protected Component HOC

```
function withAuth(WrappedComponent) {
  return function AuthenticatedComponent(props) {
    const { user } = useAuth();
    if (!user) {
      return <Navigate to='/login' />; // not logged in → redirect
    }
    // Pass all original props + inject user prop
    return <WrappedComponent {...props} user={user} />;
  };
}

// Enhance any component with auth protection
const ProtectedDashboard = withAuth(Dashboard);
const ProtectedSettings = withAuth(Settings);
const ProtectedProfile = withAuth(ProfilePage);
// In App – no need to check auth inside Dashboard itself ■
<Route path='/dashboard' element={<ProtectedDashboard />} />
```

4. withLoading — Loading State HOC

```

function withLoading(WrappedComponent) {
  return function WithLoadingComponent({ isLoading, ...props }) {
    if (isLoading) return <Spinner />; // show spinner
    return <WrappedComponent {...props} />; // show component
  };
}

const UserListWithLoading = withLoading(UserList);

function App() {
  const { data, loading } = useFetch('/api/users');
  return <UserListWithLoading isLoading={loading} users={data} />;
}

```

5. withLogger — Debug HOC

```

function withLogger(WrappedComponent) {
  return function LoggedComponent(props) {
    useEffect(() => {
      console.log(`${WrappedComponent.name} mounted`, props);
      return () => console.log(`${WrappedComponent.name} unmounted`);
    }, []);
    useEffect(() => {
      console.log(`${WrappedComponent.name} props updated`, props);
    });
    return <WrappedComponent {...props} />;
  };
}

const LoggedUserCard = withLogger(UserCard); // wrap any component to debug ■

```

6. HOC Rules — 5 You Must Know

Rule	Correct	Wrong
Name starts with 'with'	withAuth(), withLoading()	enhanceComponent() ■
Always spread original props	<WrappedComponent {...props} />	<WrappedComponent /> — props lost ■
Set displayName for DevTools	Component.displayName = 'WithAuth(X)'	Anonymous in DevTools ■
Never mutate original component	Return a NEW component always	WrappedComponent.newMethod = () => {} ■
Never create HOC inside render	const P = withAuth(D); outside comp	const P = withAuth(D); inside render ■

7. Setting displayName — DevTools Readable

```
function withAuth(WrappedComponent) {  
  function AuthComponent(props) { ... }  
  // Set display name so DevTools shows: WithAuth(Dashboard)  
  AuthComponent.displayName =  
    `WithAuth(${WrappedComponent.displayName || WrappedComponent.name})`;  
  return AuthComponent;  
}  
// React DevTools shows: WithAuth(Dashboard) ■  
// Instead of: Component (anonymous) ■
```

8. Composing Multiple HOCs

```
// Reads RIGHT to LEFT – withLogger wraps withAuth wraps withLoading(Dashboard)  
const EnhancedComponent = withLogger(withAuth(withLoading(Dashboard)));  
  
// Cleaner with compose utility  
const enhance = compose(withLogger, withAuth, withLoading);  
const EnhancedDashboard = enhance(Dashboard);  
  
// compose(f, g, h)(x) === f(g(h(x)))
```

■ **Multiple HOCs = wrapper hell in DevTools. Each adds a layer. Custom hooks compose flat — no wrapping problem.**

9. Render Props — Basic Concept

```
// Render Props = pass a FUNCTION as a prop
// Component provides LOGIC, you provide UI
// Pattern 1 – render prop
<DataProvider render={({data}) => <MyUI data={data} />} />
// Pattern 2 – children as function (more common)
<DataProvider>
  {(data) => <MyUI data={data} />}
</DataProvider>
```

■ **Component owns the logic. YOU own the rendering. Same data — unlimited different UIs.**

10. Render Prop 1 — Mouse Tracker

```
// Component provides mouse position logic
function MouseTracker({ render }) {
  const [pos, setPos] = useState({ x: 0, y: 0 });
  useEffect(() => {
    function handleMove(e) {
      setPos({ x: e.clientX, y: e.clientY });
    }
    window.addEventListener('mousemove', handleMove);
    return () => window.removeEventListener('mousemove', handleMove);
  }, []);
  return render(pos); // calls YOUR function with the data ■
}

// YOU decide what to show with that data
function App() {
  return (
    <MouseTracker
      render={({ x, y }) => (
        <div>
          <p>Mouse: {x}, {y}</p>
          <img src='/cat.png'
            style={{ position:'fixed', left:x, top:y }} />
        </div>
      )}
    />
  );
}
```

11. Render Prop 2 — Toggle (Children as Function)

```
function Toggle({ children }) {
  const [isOn, setIsOn] = useState(false);
  return children({ // calls children as a function
    isOn,
    toggle: () => setIsOn(prev => !prev),
    setOn: () => setIsOn(true),
    setOff: () => setIsOn(false),
  });
}

// YOU decide what toggle renders
function App() {
  return (
    <Toggle>
      ({ isOn, toggle }) => (
        <div>
          <button onClick={toggle}>
            {isOn ? 'Turn Off' : 'Turn On'}
          </button>
          {isOn && <p>The thing is ON!</p>}
        </div>
      )
    </Toggle>
  );
}
```

12. 3-Way Comparison — Same Logic, 3 Patterns

Mouse position logic implemented as HOC, Render Props, and Custom Hook.

```
// Way 1 – HOC (wraps component, injects prop)
function withMousePosition(WrappedComponent) {
  return function(props) {
    const [pos, setPos] = useState({ x:0, y:0 });
    useEffect(() => { /* listener */ }, []);
    return <WrappedComponent {...props} mousePos={pos} />;
  };
}

const CatWithMouse = withMousePosition(Cat);
// Usage: <CatWithMouse />

// Way 2 – Render Props (component provides logic)
function MousePosition({ render }) {
  const [pos, setPos] = useState({ x:0, y:0 });
  useEffect(() => { /* listener */ }, []);
  return render(pos);
}
// Usage: <MousePosition render={pos => <Cat pos={pos} />} />

// Way 3 – Custom Hook ■ (modern – cleanest!)
function useMousePosition() {
  const [pos, setPos] = useState({ x:0, y:0 });
  useEffect(() => { /* listener */ }, []);
  return pos;
}
```

```
function Cat() {
  const pos = useMousePosition();
  return <img style={{ left: pos.x, top: pos.y }} />;
}
// Usage: <Cat /> ← cleanest, no wrappers, no extra props ■
```

	HOC	Render Props	Custom Hook
Logic sharing	■ Yes	■ Yes	■ Yes
JSX flexibility	■ Fixed	■ Full	■ Full
Prop collision risk	■■ Yes	■ None	■ None
Nesting / wrapping	■■ Hell	■■ Cbk	■ Flat
Works without JSX	■ No	■ No	■ Yes
Modern preference	Legacy	Legacy	■ Use this

13. When Each Pattern is Still Used Today

Pattern	Still useful for	Avoid for
HOC	Third-party component wrapping, route-level concerns, library code (Redux connect, React.memo)	New application logic — hooks are cleaner
Render Props	Headless UI libs (Downshift), libraries pre-hooks (Formik v1), full render control	New code — extract to custom hook instead
Custom Hook	Everything — sharing state, effects, refs, callbacks	Nothing — this is the modern default

■ **Custom Hooks replaced both HOC and Render Props for new application code. No wrappers, no prop collisions, composable, works outside JSX.**

14. Interview Q&A;

Q

Beginner

Q1. What is a Higher Order Component?

→ A HOC is a function that takes a component and returns a new enhanced component. It's a pattern for reusing component logic — you wrap a component to add behaviour like auth checks, loading states, or logging without modifying the original component.

Q

Beginner

Q2. What are the naming conventions for HOCs?

→ By convention, HOC function names start with 'with' — like `withAuth`, `withLoading`, `withLogger`. The returned component should have a `displayName` set for React DevTools visibility. Always spread the original props through to the wrapped component so nothing is lost.

Q

Beginner

Q3. What is a Render Prop?

→ A render prop is a prop whose value is a function that returns JSX. The component provides logic — like mouse position or toggle state — and calls your function with that data. You decide what UI to render. The children-as-function pattern is the most common variant.

Q

Intermediate

Q4. What are the problems with HOCs?

→ Prop collision — HOC might inject a prop with the same name as an existing prop. Wrapper hell — composing many HOCs creates deeply nested trees in DevTools. Ref forwarding issues — refs don't pass through automatically without `forwardRef`. These problems are why custom hooks are now preferred.

Q

Intermediate

Q5. What is the difference between HOC and Render Props?

→ HOC wraps a component and injects data as props — the UI is fixed inside the wrapped component. Render Props pass a function that you call with the data, giving complete control over what renders. Both share logic; Render Props give more UI flexibility but can create callback nesting.

Q

Intermediate

Q6. When would you use a HOC today?

→ HOCs are still useful for wrapping third-party components you can't modify, applying cross-cutting concerns across many components at once, and in library code like `Redux connect()`, `React.memo`, or `forwardRef`. For new application code, custom hooks are cleaner.

Q

Expert

Q7. What replaced HOCs and Render Props?

→ Custom hooks replaced both for most use cases. Hooks allow sharing stateful logic without adding wrapper components, avoid prop name collisions, compose cleanly by just calling multiple hooks, and can be used outside JSX. The React team recommends hooks over HOCs for new code.

Q**Expert****Q8. Why should you not create a HOC inside a render function?**

→ Creating a HOC inside render creates a new component type on every render. React sees a different component type each render and unmounts + remounts it instead of updating — losing all state and causing performance issues. Always create HOC-wrapped components outside the component function.